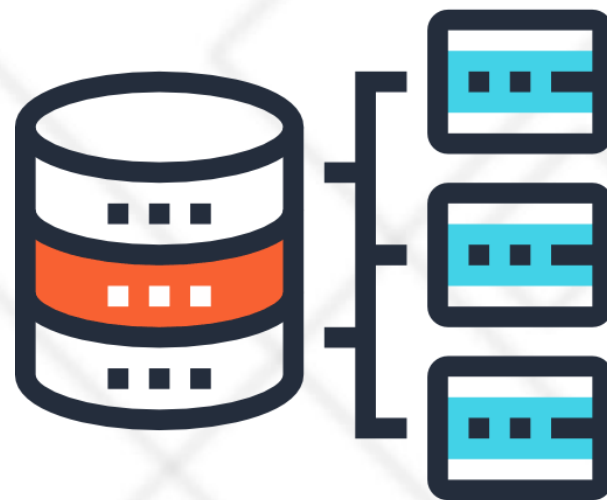




北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



数据库原理与设计 期末总结

北京理工大学 计算机学院

段春晖

duanch@bit.edu.cn

课程考核方式

□课程性质

- 专业必修课，总学分：3.0，总学时：48学时

□考核与成绩评定

- 平时成绩：30%
- 期末成绩（闭卷考试）：70%
 - ◆ 客观题：填空题、判断题、选择题
 - ◆ 主观题：简答题、综合题、设计题

- 第一部分 数据库的基本概念 (1、2)
- 第二部分 关系数据库 (3、4、5)
- 第三部分 数据控制 (6、7、8、9)
- 第四部分 数据库设计 (10、11、12)

第1章 数据库系统引论



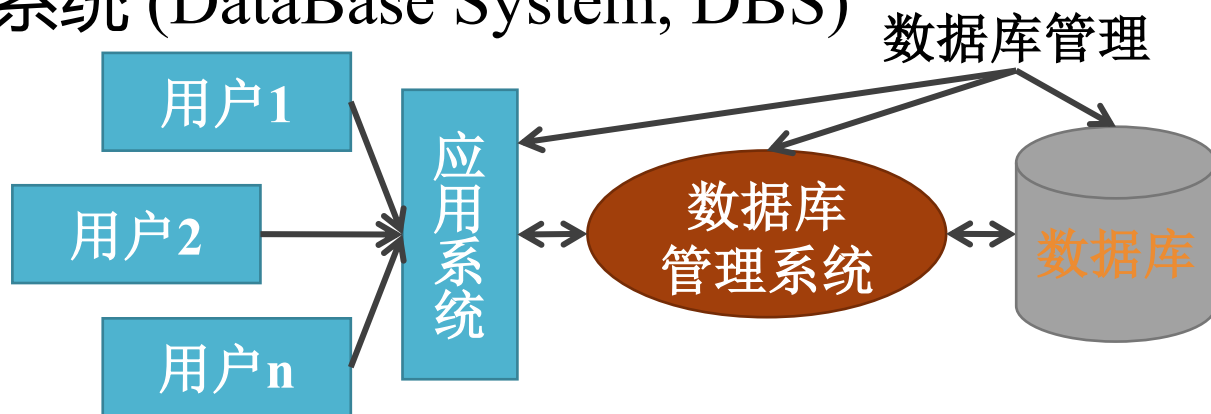
- 1. 数据管理技术的发展
- 2. 数据库基本概念
- 3. 数据模型
- 4. 数据库系统结构
- 5. 数据库管理系统

1. 数据管理技术的发展

- 人工管理阶段 (40年代中--50年代中)
- 文件系统阶段 (50年代末--60年代中)
- 数据库系统阶段 (60年代末--现在)
 - 数据结构化
 - 数据独立性高 (物理独立性、逻辑独立性)
 - 减少数据冗余
 - 数据共享
 - 统一的数据保护功能

2. 什么是数据库

- ❑ 数据库 (DataBase, DB) 是长期存储在计算机内、有组织的数据集合, 它根据数据间的联系组织在一起, 具有较高的数据独立性, 较少数据冗余, 能够为各种用户共享
- ❑ 它需要由一个软件系统统一管理, 这个软件系统称为数据库管理系统 (DataBase Management System, DBMS)
- ❑ 对数据提供存储、管理和应用的计算机系统称为数据库系统 (DataBase System, DBS)



3. 数据模型

□ 数据模型是数据特征的抽象，用来描述数据的一组概念和定义。数据模型的三个要素：

- (1) 数据结构

是区别数据模型最主要的部分

- ◆ 对数据静态特性的描述
- ◆ 应用所涉及的对象和对象具有的特征，对象间的联系

- (2) 数据操作

- ◆ 对数据的动态特性的描述。主要分为更新和检索两大类
- ◆ 具体包括检索、插入、删除、修改等

- (3) 数据的完整性约束

- ◆ 对数据静态和动态特性的限定，反映了数据间的制约和依存关系

4. 数据库系统结构

□ 数据库系统结构的三级模式

- 外模式：

- ◆ 是模式的子集，是用户的数据视图

- 模式（也称逻辑模式）：

- ◆ 是全体数据的逻辑结构和特征的描述，独立于应用程序和物理存储

- 内模式：

- ◆ 是数据物理结构和存储方式的描述，是数据在数据库内部的表示方法

4. 数据库系统结构

□三级模式结构的二级映像

- 目的：实现三个模式的联系和转换

□外模式/模式映像

- 一个模式对应多个外模式，当模式结构改变，则只要修改外模式与模式间的映像关系，而不必修改外模式中的局部逻辑结构，因而相应的应用程序亦可不必修改，实现了数据的逻辑独立性

□模式/内模式映像

- 一个模式对应一个内模式。当数据库的物理存储结构改变时，仅需要修改模式与内模式间的映像关系，而可以使模式保持不变，从而使应用程序保持不变，提供了数据的物理独立性



5. 数据库管理系统

- 数据库的定义功能
- 数据库的操纵功能
- 数据库的保护功能
- 数据库维护功能

第2章 数据模型



- 1. E-R概念模型
- 2. 层次数据模型
- 3. 网状数据模型
- 4. 关系数据模型

第2章 数据模型

□数据模型是数据库研究的一个核心问题

□概念模型

- 按用户的观点来对数据和信息建模。不涉及信息在计算机中如何表示；如实体-联系模型

□逻辑模型

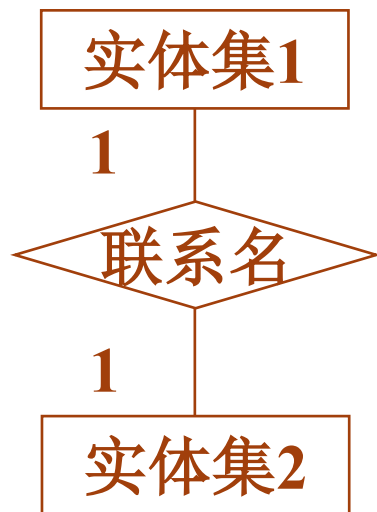
- 按计算机系统的观点对数据建模主要包括网状模型、层次模型、关系模型、面向对象模型、对象关系模型等

□物理模型

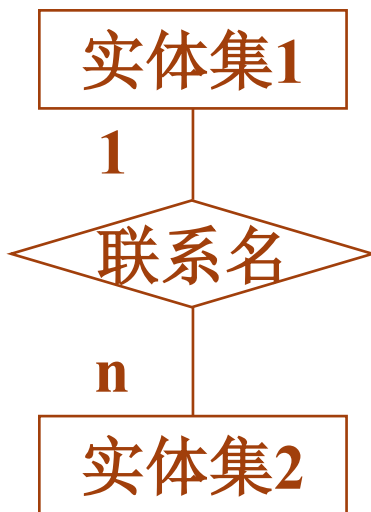
- 它描述数据在系统内部的表示方式和存取方法，在磁盘或磁带上的存储方式和存取方法

1. E-R概念模型

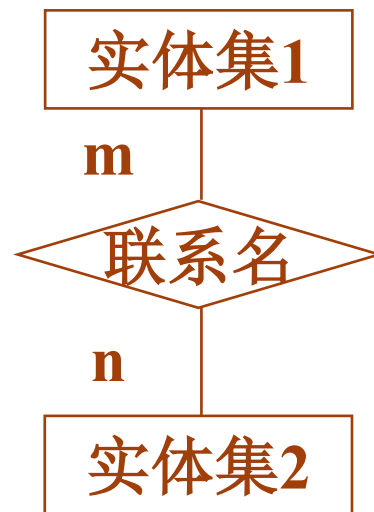
- (1) 实体：客观存在并可相互区别的事物
- (2) 属性：实体所具有的某一特征
- (3) 联系：在现实世界中，事物内部以及事物之间是有联系的
 - 两个实体集之间的三类联系



1:1联系



1:n联系

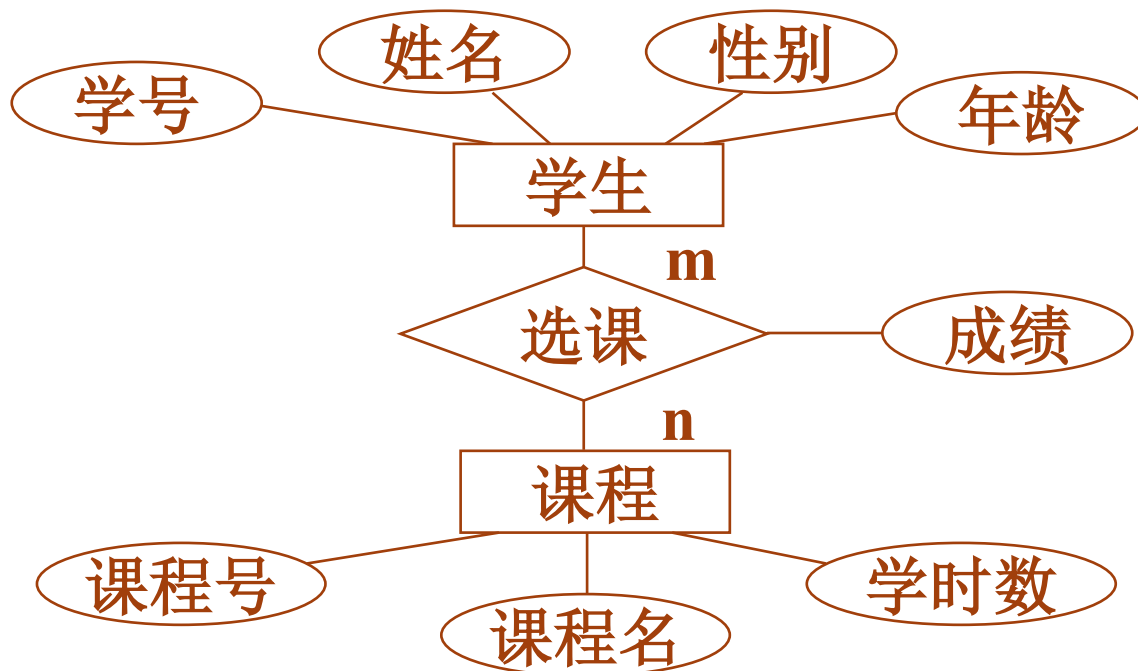


m:n联系

1. E-R概念模型

□ E-R图三个基本成分：实体、属性和联系

- (1) 实体：用矩形表示，矩形框内写明实体名
- (2) 属性：用椭圆形表示
- (3) 联系：实体之间的联系用菱形框表示





第2章 数据模型



□2. 层次模型

- 层次数据模型 (hierarchical data model) 是较早用于数据库技术的一种数据模型，它是按层次结构来组织数据的

□3. 网状数据模型

- 网状模型的结点间的联系可以是任意的，任何二个结点间都能发生联系，更适于描述客观世界
- 将层次模型中对结点的限制去掉后就成为网状模型

□4. 关系数据模型

- 在关系模型中，基本数据结构是二维表
- (1) 关系 (relation)
 - ◆ 关系是一张二维表，是由多个行和列组成的。一个关系可用来描述一个实体集
- (2) 属性 (attribute)
- (3) 域 (domain)
- (4) 元组 (tuple)
- (5) 键 (key)
 - ◆ 键是一个或多个属性组成的，能够唯一标识一个元组
 - ◆ 一个关系中可能有多组属性都能够起到标识元组的作用。因而，一个关系中可能有多个键。选择其中的一个作为主键，其余为候选键



第3章 关系数据库

- 1. 关系模型的基本概念
- 2. 关系代数
- 3. 关系演算
 - 域关系演算
 - 元组关系演算

1. 关系模型的基本概念

□**定义3.1** 给定一组集合 $D_1, D_2, \dots, D_n$, 它们可以是相同的。 $D_1, D_2, \dots, D_n$ 的笛卡尔积为:

$$D_1 \times D_2 \times \dots \times D_n = \{(d_1, d_2, \dots, d_n) \mid d_i \in D_i, i=1, 2, \dots, n\}$$

- 所有域的所有值的一个组合, 不能重复

□**定义3.2** $D_1 \times D_2 \times \dots \times D_n$ 的任一个子集称为 $D_1, D_2, \dots, D_n$ 上的一个关系。 n 叫做关系的目或度 (degree)

1. 关系模型的基本概念

- 无限关系在数据库中是无意义的，因此限定关系代数数据模型中的关系必须是有限集合
- 关系数据库中，关系都是规范化的，具有如下性质：
 - (1) 每一列中的值是同类型的数据，来自同一个域
 - (2) 不同的列可以有相同的域，每一列称为属性，用属性名标识
 - (3) 列的次序是无关紧要的
 - (4) 元组的每个分量是原子的，是不可分的数据项
 - (5) 元组的次序是无关紧要的
 - (6) 各个元组是不同的，即关系中不允许出现重复元组

1. 关系模型的基本概念

□ 对关系结构的描述称为关系模式。用如下形式表示：

关系名(属性名1, 属性名2, ..., 属性名n)

□ 关系模型的数据完整性约束

- 实体完整性、参照完整性、用户自定义完整性
- 其中，实体完整性、参照完整性是必须支持的

□ 关系模型的数据操纵

- 查询、插入、删除和修改操作
- 在关系数据库系统中，对数据的全部操作都可以归结为对关系的运算

2. 关系代数

□ 关系代数运算的分类：

- 传统的集合运算

- ◆ 并、差、交、笛卡尔积

- 专门的关系运算

- ◆ 选择、投影、连接、除

- ◆ 不仅涉及行运算，也涉及列运算，这种运算是为数据库的应用而引进的特殊运算

- 关系代数中五种基本运算

- ◆ 并、差、笛卡尔积、投影、选择

- 交、连接、除法可以用5种基本运算来表达

- 引进它们并不增加语言的能力，但可以简化表达

2. 关系代数

- 集合运算是二个关系间的运算，运算结果生成一个新关系。其中，并($\cup$)、交($\cap$)、差($-$) 运算要求
 - 参加运算的二个关系 R 和 S 具有相同的目
 - 且其对应属性定义在同一个域上
 - 称 R 和 S 为同类型的关系

□ $R \cup S$

- 仍为 n 目关系，由属于 R 或属于 S 的元组组成

$$R \cup S = \{ t \mid t \in R \vee t \in S \}$$

□ $R - S$

- 仍为 n 目关系，由属于 R 而不属于 S 的所有元组组成

$$R - S = \{ t \mid t \in R \wedge t \notin S \}$$

2. 关系代数

□ $R \cap S$

- 仍为 n 目关系，由既属于 R 又属于 S 的元组组成

$$R \cap S = \{ t \mid t \in R \wedge t \in S \}, \quad R \cap S = R - (R - S)$$

□ $R \times S$

- 关系 R 和 S 的笛卡尔积为 R 中所有元组和 S 中所有元组的拼接

□ $\sigma_F(R)$

- 选择运算是关系上的一元运算，是从关系中选择满足一定条件的元组子集

$$\sigma_F(R) = \{ t \mid t \in R \wedge F(t) = \text{'真'} \}$$

2. 关系代数

□ $\Pi_A(R)$

- 从 R 中选择出若干属性列组成新的关系

□ $R \bowtie_{A \theta B} S = \sigma_{A \theta B} (R \times S)$

- 条件连接是将二个关系中满足连接条件的元组拼接起来形成新元组的集合，也叫 θ 连接
- 自然连接是在两个关系共同属性上的等值连接
 - ◆ 它要求两个关系中进行比较的分量必须是相同的属性组，并且在结果中把重复的属性列去掉

第4章 关系数据库标准语言SQL



- 1. 数据定义：CREATE
- 2. 数据查询：SELECT
- 3. 数据更新：INSERT, UPDATE, DELETE
- 4. SQL中的视图
- 5. SQL的数据控制
- 6. 嵌入式SQL

第4章 关系数据库标准语言SQL



- 结构化查询语言SQL (Structured Query Language) 是一种介于关系代数与关系演算之间的语言，是一个通用的、功能极强的关系数据库语言，是关系数据库的标准语言
- SQL语言集数据定义语言DDL、数据查询语言DQL、数据操纵语言DML、数据控制语言DCL的功能于一体，充分体现了关系数据库语言的特点和优点

1. 数据定义

□ SQL的数据定义功能主要包括定义表、视图和索引和数据库模式的定义

表 SQL 的数据定义语句

操 作 对 象	操 作 方 式		
	创 建	删 除	修 改
模 式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视 图	CREATE VIEW	DROP VIEW	
索 引	CREATE INDEX	DROP INDEX	

□表的定义、删除与修改， CREATE TABLE、ALTER TABLE、DROP TABLE



□索引类型

- 依据索引的顺序和数据库的物理存储顺序是否相同，索引分为两类：**聚集索引**、非聚集索引
- 唯一索引、组合索引等

□数据查询

- 单表查询、连接查询（外连接）
- 嵌套查询、集合查询

2. 数据查询

□ 语句格式

SELECT [ALL|DISTINCT] <目标列表达式> [, <目标列表达式>] ...

FROM <表名或视图名>[, <表名或视图名>] ...

[**WHERE** <条件表达式>]

[**GROUP BY** <列名1> [**HAVING** <条件表达式>]]

[**ORDER BY** <列名2> [ASC|DESC]];

- **SELECT子句**：指定要显示的属性列
- **FROM子句**：指定查询对象（基本表或视图）
- **WHERE子句**：指定查询条件
- **GROUP BY子句**：对查询结果按指定列的值分组，该属性列值相等的元组为一个组。通常会在每组中作用聚集函数
- **HAVING短语**：筛选出只有满足指定条件的组
- **ORDER BY子句**：对查询结果表按指定列值的升序或降序排序

使用聚集函数

□ SQL提供了多种聚集函数，用来实现统计查询

- 计数

- ◆ COUNT([DISTINCT|ALL] *)

- ◆ COUNT([DISTINCT|ALL] <列名>)

- 计算总和 SUM([DISTINCT|ALL] <列名>)

- 计算平均值 AVG([DISTINCT|ALL] <列名>)

- 求最大值 MAX([DISTINCT|ALL] <列名>)

- 求最小值 MIN([DISTINCT|ALL] <列名>)

□ DISTINCT短语：在计算时要取消指定列中的重复值

□ ALL短语：不取消重复值；ALL为缺省值

对查询结果分组GROUP BY



□ 细化聚集函数的作用对象

- 未对查询结果分组，聚集函数将作用于整个查询结果
- 对查询结果分组后，聚集函数将分别作用于每个组

□ 分组方法：按指定的一列或多列值分组，值相等的为一组

□ 使用GROUP BY子句后，SELECT子句的列名列表中只能出现分组属性和聚集函数

对查询结果分组GROUP BY



- GROUP BY子句的作用对象是查询的中间结果表
- 使用HAVING短语筛选最终输出结果
 - 只有满足HAVING短语指定条件的组才输出

[例] 查询选修了3门以上课程的学生学号

SELECT Sno

FROM SC

GROUP BY Sno

HAVING COUNT(*)>3;

3. 数据更新

插入数据

□(1) 插入单个元组

- 语句格式

**INSERT INTO <表名> [(<属性列1>[, <属性列2 > ...])]
VALUES (<常量1> [, <常量2>] ...);**

□(2) 插入子查询结果

- 语句格式

**INSERT INTO <表名> [(<属性列1>[, <属性列2 > ...])]
<SELECT语句>;**

3. 数据更新

□ 修改数据

UPDATE <表名>

SET 列名1=<表达式1>[, 列名2=<表达式2>] ...

[WHERE <条件表达式>];

□ 删除数据

DELETE FROM <表名>

[WHERE <条件>];

4. SQL中的视图

□视图是建立在一个或多个基本表上的虚表。视图提供了用户从不同角度观察数据库的方法

□视图的定义

CREATE VIEW <视图名> [(<列名> [, <列名>]...)]

AS <子查询>

[WITH CHECK OPTION];

- WITH CHECK OPTION：通过视图进行增删改操作时，不得破坏视图定义中子查询中的条件表达式

□视图的删除

DROP VIEW <视图名>;

- 该语句从数据字典中删除指定的视图定义

4. SQL中的视图

□更新视图

- 可以通过视图插入、删除和修改数据，对视图的更新最终要转换成对基本表的更新

□DBMS实现视图查询的方法

- 实体化视图、视图消解法
- 视图的更新操作有些可以执行；有些不能执行，即不能转换为对基本表的对应操作

□视图的优点

- (1) 视图提供了逻辑数据独立性
- (2) 简化了用户视图
- (3) 视图使用户以不同角度看待相同的数据
- (4) 视图提供了安全保护功能



□5. SQL的数据控制

- 授权语句GRANT
- 回收语句REVOKE
 - ◆ 具有授予权的用户可以通过回收语句REVOKE将所授予的权限回收

□6. 嵌入式SQL

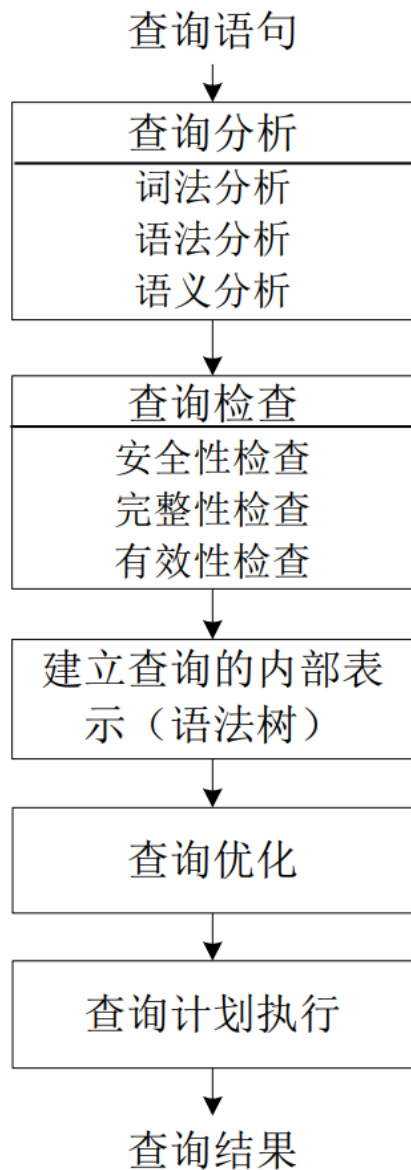
- SQL语言提供了两种使用方式：
 - ◆ 交互式SQL、嵌入式SQL
- 嵌入式SQL引入了**游标**机制协调面向集合和面向记录两种不同的处理方式

第5章 查询处理和查询优化



- 1. 查询处理
- 2. 查询优化
- 3. 代数优化
- 4. 基于存取路径的优化
- 5. 基于代价估算的优化

1. 查询处理



□ 查询处理过程包括：

- 查询分析
- 查询检查
- 建立查询的内部表示
- 查询优化
- 查询执行

1. 查询处理

□(1) 选择操作的实现

- 顺序扫描方法
- 二分查找法
- 使用索引（或散列）的扫描方法

□(2) 连接操作的实现

- 嵌套循环法
- 索引嵌套循环法
- 排序合并法
- 散列连接

2. 查询优化

□ 查询优化的总目标

- 选择有效策略使查询代价最小（实际上是较小）
- (1) 代数优化
 - ◆ 是关系代数表达式的优化，按照一定的规则，改变代数表达式中操作的次序和组合，使查询执行更高效。不涉及底层的存取路径
- (2) 基于存取路径的优化
 - ◆ 合理选择各种操作的存取路径以获得优化效果，需要考虑数据的物理组织和访问路径，以及底层操作算法的选择，涉及数据文件的组织方法、数据值的分布情况等，也称为物理优化
- (3) 基于代价估算的优化
 - ◆ 对于多个可选的查询策略，通过估算执行策略的代价，从中选择代价最小的作为执行策略

2. 查询优化

□代数优化策略

- (1) 在关系代数表达式中尽可能早地执行选择操作，这是最重要、最基本的一条
- (2) 投影运算和选择运算同时进行
- (3) 将投影运算与其前面或后面的双目运算结合
- (4) 把某些选择同在它前面要执行的笛卡尔积结合起来成为一个连接运算
- (5) 找出公共子表达式

2. 查询优化

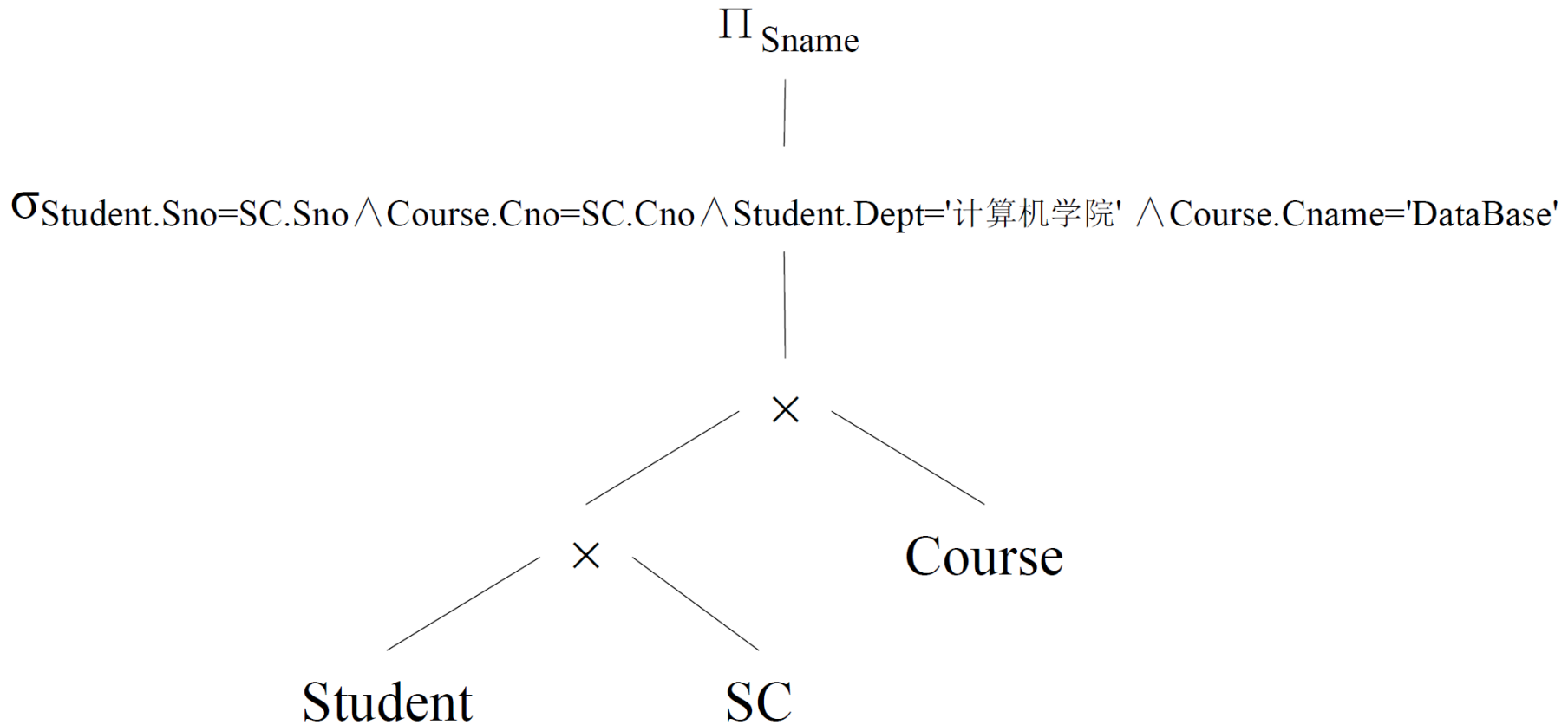
□代数优化算法

- 遵循代数优化的启发式规则，应用关系代数等价变换公式来优化关系表达式的算法

- (1) 把形如 $\sigma_{F1 \wedge F2 \wedge \dots \wedge F_n}(E)$ 变换为 $\sigma_{F1}(\sigma_{F2}(\dots(\sigma_{F_n}(E))\dots))$
- (2) 对每一个选择，尽可能把它移到树的叶端
- (3) 对每一个投影，尽可能把它移向树的叶端
- (4) 利用等价变换规则把选择和投影的串接合并，使多个选择或投影能同时执行，或在一次扫描中全部完成
- (5) 把语法树的内节点分组。每一双目运算和它所有的直接祖先为一组

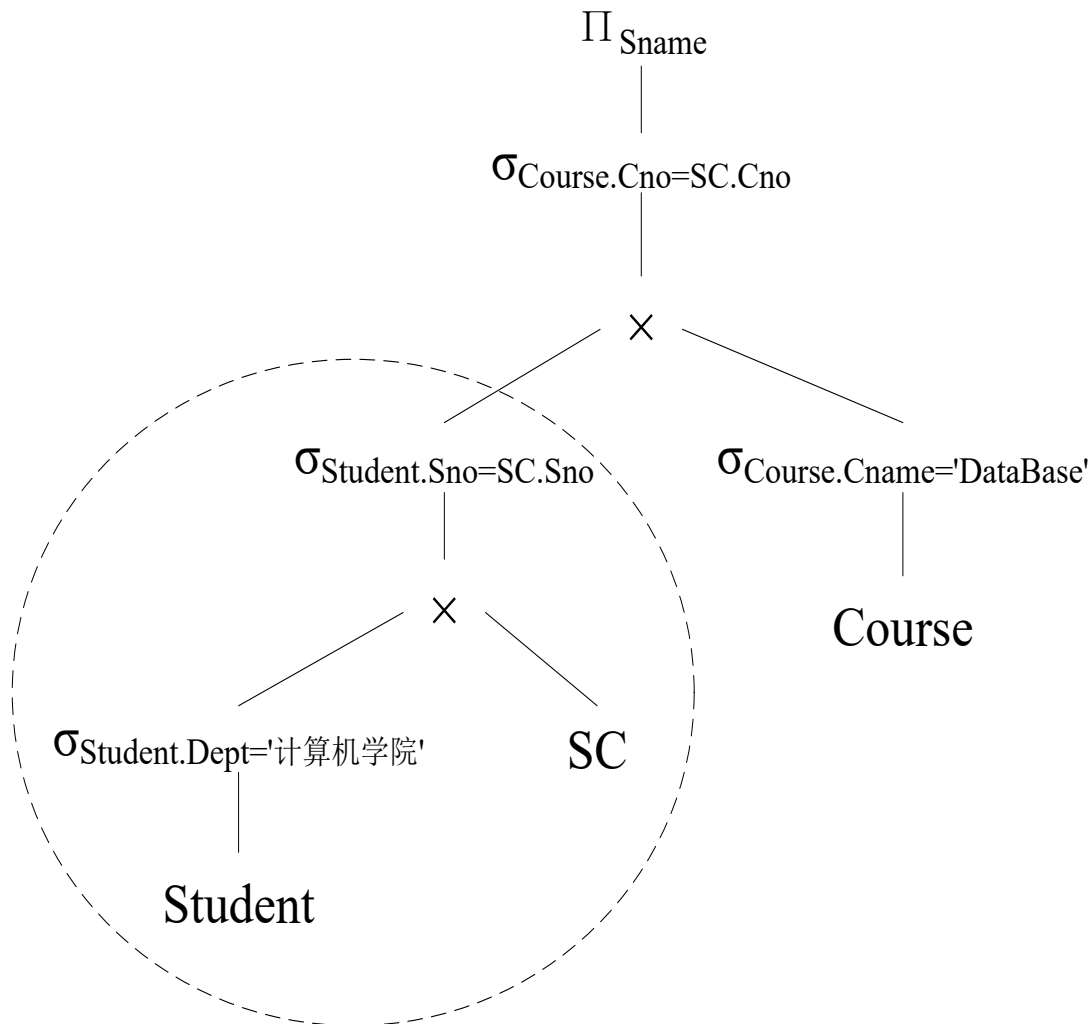
2. 查询优化

□代数优化算法



2. 查询优化

□ 代数优化算法



第6章 数据库的安全性

数据库中所采用的安全性方法和技术

□1. 用户标识与鉴别：

- 该方法由系统提供一定的方式让用户标识自己的名字或身份。用户进入系统时，由系统进行核对，通过鉴定后才提供系统的使用权
- 最外层的安全保护措施

□2. 存取控制：

- 通过用户权限定义和合法权检查确保只有合法权限的用户访问数据库，所有未被授权的人员无法存取数据

第6章 数据库的安全性



□3. 视图机制：

- 为不同的用户定义视图，通过视图机制把要保密的数据对无权存取的用户隐藏起来，从而自动地对数据提供一定程度的安全保护

□4. 数据加密：

- 对存储和传输的数据进行加密处理，从而使得不知道解密算法的人无法获知数据的内容

□5. 数据库审计：

- 数据库审计可以作为预防手段，建立审计日志，把用户对数据库的所有操作自动记录下来放入审计日志中，DBA 可以利用审计跟踪的信息，重现导致数据库现有状况的一系列事件，找出非法存取数据的人、时间和内容等

2. 存取控制

□(1) 自主存取控制DAC

- **主体**是指一个提出请求或要求的实体，主体可以是DBMS所管理的实际用户，或其它任何代表用户行为的进程、作业和程序
- **客体**是接受其他实体访问的被动实体，是受主体操纵，客体可以是文件、记录、视图等
- **控制策略**是主体对客体的操作行为集和约束条件集，即主体对客体的访问规则集
- 通过SQL的GRANT语句和REVOKE语句实现

2. 存取控制

□(2) 强制存取控制MAC

- 每一个数据对象被标以一定的密级，每一个用户也被授予某一个级别的许可证。对于任意一个对象，具有合法许可证的用户才可以存取
- 强制存取控制规则：
 - ◆ 1) 仅当主体的许可证级别大于等于客体的密级时，该主体才能读取相应的客体
 - ◆ 2) 仅当主体的许可证级别小于等于客体的密级时，该主体才能写相应的客体

第7章 数据库的完整性

- 数据的完整性是指数据的正确性、有效性和相容性
- 为维护数据库的完整性，DBMS必须提供哪些功能？
 - 提供定义完整性约束条件的机制
 - 提供完整性检查的方法
 - 违约处理
 - ◆ 即如果发现用户的操作请求使数据违背了完整性约束条件，则采取一定的动作来保证数据的完整性
- 完整性约束条件的作用对象可以是：
 - 关系、元组和属性（列）

1. 实体完整性约束

- 实体完整性是指主键的值不能为空或部分为空
 - 如果一个元组的键为空值，或部分为空，该元组将不能表示任何实体，因而无意义
 - 通过对主键值的约束实现实体完整性

CREATE TABLE SC

(Sno CHAR(9) NOT NULL,

Cno CHAR(4) NOT NULL,

Grade SMALLINT,

PRIMARY KEY (Sno, Cno) /*只能在表级定义主键*/

);

2. 参照完整性约束

- 参照完整性约束是对关系中作为外键的值的约束
 - 若关系R1中属性A定义为外键，参照关系R2中的主键，则对于关系R1中的任一个元组在属性A上的值或者为空值，或者为另一个关系R2中某个元组的主键的值
 - 用 FOREIGN KEY 短语定义哪些列为外键，用 REFERENCES 短语指明外键参照哪些表的主键

CREATE TABLE SC

(Sno CHAR(9) NOT NULL,

Cno CHAR(4) NOT NULL,

Grade SMALLINT,

PRIMARY KEY (Sno, Cno),

FOREIGN KEY (Sno) REFERENCES Student(Sno),

FOREIGN KEY (Cno) REFERENCES Course(Cno));

2. 参照完整性约束

□ 参照完整性违约处理

- 拒绝执行 (NO ACTION)
- 级联操作 (CASCADE)
- 设置为空值 (SET-NULL)

3. 用户定义的完整性

- 用户定义的完整性反映应用领域需要遵循的约束条件，体现了具体领域中的语义约束，用户定义后由系统支持
- 在关系数据库系统中，数据完整性控制策略包括规则、默认值、约束、触发器和存储过程等
- 通常属性上的约束条件包括
 - 唯一约束 (UNIQUE)
 - 非空约束 (NOT NULL)
 - 默认值约束 (DEFAULT)
 - 检查约束 (CHECK)

第7章 数据库的完整性

- 表的设计要体现完整性约束的实现
 - 实体完整性的体现是主键约束
 - 外键约束是参照完整性的体现
 - 默认值和唯一等是用户定义完整性的体现

4. 触发器

- 触发器是用户定义在关系数据表上的一类由事件驱动的特殊过程，用编程的方法实现复杂的业务规则
- 触发器工作过程
 - inserted表
 - ◆ 存放insert或update语句执行过程中，插入到触发表中的新数据行的副本。因此inserted表中的行是和触发表中的新数据行相同
 - deleted表
 - ◆ 存放delete或update语句执行过程中，从触发表中删除的旧数据行的副本。因此，deleted表和触发表不会有相同的行

第8章 数据库恢复技术



- 1. 事务的基本概念
- 2. 故障种类
- 3. 数据库恢复技术
- 4. 数据库恢复策略
- 5. 检查点恢复技术

1. 事务的基本概念

- 事务 (Transaction) 是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位
- **原子性**：事务是数据库的逻辑工作单位，事务中包括的诸操作要么都做，要么都不做
- **一致性**：事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态
- **隔离性**：对并发执行而言，一个事务的执行不能被其他事务干扰
- **持续性**（也称永久性）：一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。接下来的其他操作或故障不应该对其执行结果有任何影响

2. 故障种类

□数据库系统可能发生的故障大致分为以下几类：

- 系统故障

- ◆ 整个系统的正常运行突然被破坏、所有正在运行的事务都非正常终止、内存中数据库缓冲区的信息全部丢失、外部存储设备上的数据未受影响

- 事务内部的故障

- ◆ 某个事务在运行过程中由于种种原因未运行至正常终止点就夭折了

- 介质故障

- 其它原因

3. 数据库恢复技术

□ 数据恢复的基本原理

- **数据冗余**：利用存储在系统其它地方的冗余数据来重建数据库中已被破坏或不正确的那部分数据

□ 建立冗余数据最常用的技术是

- **数据转储和登记日志文件**
- 通常在一个数据库系统中，这两种方法一起使用

□ **恢复子系统**的功能是利用冗余数据，再根据故障的类型采取相应的恢复措施，保证故障发生后，能把数据库中的数据从错误状态恢复到某种逻辑一致的状态

3. 数据库恢复技术

□ 恢复中经常使用的技术

1) 数据转储

- 静态转储与动态转储
- 海量转储与增量转储

2) 基于日志的恢复技术

● 写日志文件的两条原则

- ◆ 登记的次序严格按并行事务执行的时间次序
- ◆ 必须先写日志文件，后写数据库

● REDO技术：重做已提交的事务

● UNDO技术：对没有提交的事务执行UNDO，将被修改的数据项恢复到事务开始前的状态

3. 数据库恢复技术

- 提高恢复效率的技术
 - 检查点技术
 - 镜像技术

4. 数据库恢复策略

□ 事务故障的恢复

- 利用日志文件撤销事务对数据的更改，系统回滚到事务执行前的状态
- 当事务故障发生时，系统反向扫描日志文件，并执行逆向操作，将更新前的数据写入数据库

□ 介质故障的恢复

- 首先根据后备副本重建数据库
- 然后利用运行日志重做该副本备份后已完成的所有事务

4. 数据库恢复策略

□ 系统故障的恢复

(1) 正向扫描日志文件（即从头扫描日志文件）

- ◆ Redo队列：在故障发生前已经提交的事务
- ◆ Undo队列：故障发生时尚未完成的事务

(2) 对Undo队列事务进行UNDO处理

- ◆ 反向扫描日志文件，对每个UNDO事务的更新操作执行逆操作

(3) 对Redo队列事务进行REDO处理

- ◆ 正向扫描日志文件，对每个REDO事务重新执行登记的操作

5. 检查点恢复技术

- 检查点技术可以提高系统故障恢复的效率，数据库恢复时，利用检查点能判定哪些事务是正常结束的，从而确定恢复哪些数据以及如何恢复
- 检查点记录的内容
 - 1. 建立检查点时刻所有正在执行的事务清单
 - 2. 这些事务最近一个日志记录的地址
- 写检查点的步骤
 - 把日志缓冲区中的内容写入日志文件
 - 在日志文件中写入一个检查点记录
 - 把数据库缓冲区的内容写入数据库
 - 把检查点记录在日志文件中的地址写入重启动文件

5. 检查点恢复技术

□ 利用检查点的恢复步骤

- (1) 从重启动文件中找到最后一个检查点记录
- (2) 得到检查点时刻的事务清单；把事务清单中的事务列表暂时放入Undo队列
- (3) 从检查点开始正向扫描日志文件
 - ◆ 如有新开始的事务，暂时放入Undo队列
 - ◆ 如有提交事务 T_j ，把 T_j 从Undo队列移到Redo队列
- (4) 对Undo队列事务进行UNDO处理；对Redo队列事务进行REDO处理

第9章 并发控制

□ 并发操作带来的数据不一致性

- 1. 丢失更新 (Lost Update)
- 2. 不可重复读 (Non-Repeatable Read)
- 3. 读“脏”数据 (Dirty Read)

□ 避免不一致性的方法和技术就是并发控制。最常用的技术是封锁技术

1. 并发调度的可串行性

- 定义9.1 多个事务的并发执行是正确的，当且仅当并发执行的结果与这些事务按某一串行顺序执行的结果相同，这种调度策略被称为**可串行化调度**。**可串行化是并发事务正确调度的准则**
- 定义9.2 如果一个调度 S 能通过一系列非冲突操作执行顺序的交换变成调度 S_1 ，则称**调度 S 和 S_1 冲突等价**
 - 冲突操作是指不同的事务对同一个数据的读写操作和写写操作，其他操作是不冲突操作
 - 不同事务的冲突操作和同一事务的两个操作不能交换

调度的冲突等价性

□可串行化调度的充分条件

- 一个调度 S 在保证冲突操作的次序不变的情况下，通过交换两个事务不冲突操作的次序得到另一个调度 S' ，如果 S' 是串行的，称调度 S 为冲突可串行化的调度

调度的冲突等价性

□[例 9-3] 证明调度 S 是否是可串行化调度

- $S = R_1(A)W_1(A)R_2(A)W_2(A)R_1(B)W_1(B)R_2(B)W_2(B)$
- 把 $W_2(A)$ 与 $R_1(B)W_1(B)$ 交换, 得到:

$R_1(A)W_1(A)R_2(A)R_1(B)W_1(B)W_2(A)R_2(B)W_2(B)$

T_1	T_2
Read(A)	
Write(A)	
	Read(A)
	Write(A)
Read(B)	
Write(B)	
	Read(B)
	Write(B)

调度的冲突等价性

□[例 9-3] 证明调度 S 是否是可串行化调度

- $S=R_1(A)W_1(A)R_2(A)W_2(A)R_1(B)W_1(B)R_2(B)W_2(B)$
- 把 $W_2(A)$ 与 $R_1(B)W_1(B)$ 交换, 得到:

$$R_1(A)W_1(A)R_2(A)R_1(B)W_1(B)W_2(A)R_2(B)W_2(B)$$

- 再把 $R_2(A)$ 与 $R_1(B)W_1(B)$ 交换:

$$L=R_1(A)W_1(A)R_1(B)W_1(B)R_2(A)W_2(A)R_2(B)W_2(B)$$

- 因为 L 等价于一个串行调度 T_1, T_2
- 所以调度 S 是可串行化的调度

L 和 S 是 (冲突) 等价的

2. 基于封锁的并发控制技术

□封锁的类型

- 共享锁 (Share lock, 简记为S锁, 又称为读锁)
 - ◆ 若事务T对数据对象A加上S锁, 则其它事务只能再对A加S锁, 而不能加X锁, 直到T释放A上的S锁
- 排它锁 (eXclusive lock, 简记为X锁, 又称为写锁)
 - ◆ 若事务T对数据对象A加上X锁, 则只允许T读取和修改A, 其它任何事务都不能再对A加任何类型的锁, 直到T释放A上的锁

2. 基于封锁的并发控制技术

□ 封锁协议

- 在给数据对象加锁时，要考虑何时请求锁、持有锁的时间和何时释放等，要遵从一定规则。这些规则被称为封锁协议

□ 一级封锁协议

- 事务 T 在修改数据 A 前必须先对其加 X 锁，直到事务结束才释放
- 可以避免丢失修改
- 不能保证可重复读和不读“脏”数据

2. 基于封锁的并发控制技术

□二级封锁协议

- 在一级封锁协议基础上，事务 T 在读数据 A 之前必须先对其加 S 锁，读完后即可释放 S 锁
- 可以避免：丢失修改、读“脏”数据
- 不能保证避免不可重复读的问题

□三级封锁协议

- 在二级封锁协议基础上，某一事务施加的 S 锁要保持到该事务结束时才释放
- 三级封锁协议可防止丢失修改、读“脏”数据和不可重复读

2. 基于封锁的并发控制技术

□三个协议的主要区别

- 什么操作需要申请封锁
- 何时释放锁（即持锁时间）

	X锁		S锁		一致性保证		
	操作结束释放	事务结束释放	操作结束释放	事务结束释放	不丢失修改	不读“脏”数据	可重复读
一级封锁协议		✓			✓		
二级封锁协议		✓	✓		✓	✓	
三级封锁协议		✓		✓	✓	✓	✓

2. 基于封锁的并发控制技术

- 封锁技术可以有效地解决并行操作的一致性问题，但也带来一些新的问题
 - 活锁：指某个事务由于请求封锁，但总也得不到锁而长时间处于等待状态
 - 避免方法：采用先来先服务的策略
 - 当多个事务请求封锁同一数据对象时，按请求封锁的先后次序对这些事务排队
 - 该数据对象上的锁一旦释放，首先批准申请队列中第一个事务获得锁

2. 基于封锁的并发控制技术

- 封锁技术可以有效地解决并行操作的一致性问题，但也带来一些新的问题 (续)
 - 死锁：指在同时处于等待状态的两上或多个事务中相互封锁了对方请求的资源，使得没有任何一个事务可以获得足够的资源运行完毕，而永远等待下去
 - 预防死锁方法：一次封锁法、顺序封锁法、事务重试法
 - 允许死锁发生，一旦检测到死锁，解除死锁
 - ◆ 解除死锁的方法：选择一个处理死锁代价最小的事务，将其撤销，释放此事务持有的所有的锁，使其它事务能继续运行下去

两阶段封锁协议

- 两阶段封锁协议 (Two-Phase Locking, 简称2PL) 是最常用的一种封锁协议, 理论上可以证明使用两段封锁协议产生的是可串行化调度
- 指所有事务必须分两个阶段对数据项加锁和解锁
 - 第一阶段是获得封锁, 也称为扩展阶段
 - ◆ 在这阶段, 事务可以申请获得任何数据项上的任何类型的锁, 但是不能释放任何锁
 - 第二阶段是释放封锁, 也称为收缩阶段
 - ◆ 在这阶段, 事务可以释放任何数据项上的任何类型的锁, 但是不能再申请任何锁
- 两阶段封锁协议是保证并发调度可串行性的封锁协议

3. 多粒度封锁

- **多粒度封锁**：在一个系统中同时支持多种封锁粒度供不同的事务选择
- **引进意向锁 (Intention Lock) 目的**
 - 提高对某个数据对象加锁时系统的检查效率
- **什么是意向锁**
 - 对任一结点加基本锁，必须先对它的上层结点加意向锁



常用意向锁



□意向共享锁 (Intent Share Lock, 简称IS锁)

- 如果对一个数据对象加IS锁, 表示它的后裔结点拟(意向)加S锁。例: 要对某个元组加S锁, 则要先对关系和数据库加IS锁

□意向排它锁 (Intent Exclusive Lock, 简称IX锁)

- 如果对一个数据对象加IX锁, 表示它的后裔结点拟(意向)加X锁。例: 要对某个元组加X锁, 先要对关系和数据库加IX锁

□共享意向排它锁 (Share Intent Exclusive Lock, 简称SIX锁)

- 如果对一个数据对象加SIX锁, 表示对它加S锁, 再加IX锁, 即 $SIX = S + IX$ 。例: 对某个表加SIX锁, 则表示该事务要读整个表(所以要对该表加S锁), 同时会更新个别元组(所以要对该表加IX锁)

第10章 关系数据库设计理论



□ 关系模型的存储异常

- 数据冗余
 - ◆ 大量的数据冗余不仅造成存储空间的浪费，而且存在着潜在的数据不一致
- 插入异常
- 删除异常
- 更新异常

第10章 关系数据库设计理论



□ 1. 函数依赖的定义

□ **定义10.1** 设关系模式 $R(U)$, $X, Y \subseteq U$, r 是 $R(U)$ 上的任一关系。对任意元组 $t_1, t_2 \in r$, 如果 t_1, t_2 在 X 上的属性值相等, t_1, t_2 在 Y 上的属性值亦相等

- 则称 X 函数决定 Y , 或 Y 函数依赖于 X , 记为FD $X \rightarrow Y$
- 称 X 为决定因素, 或称 X 为函数依赖的左部
- 称 Y 为函数依赖的右部

□ 平凡函数依赖与非平凡函数依赖

□ 完全函数依赖与部分函数依赖

□ 传递函数依赖与直接函数依赖

函数依赖的蕴涵性

- 若关系模式R上的任一关系都能满足一个确定的函数依赖集F，则称F为R满足的**函数依赖集**
- **定义10.5** 设函数依赖集F和关系模式R(U)，属性集 $X, Y \subseteq U$ ，关系模式R满足F。如果关系模式R满足FD $X \rightarrow Y$ ，则称F**逻辑蕴涵**FD $X \rightarrow Y$ ，或称 $X \rightarrow Y$ **逻辑蕴涵于**F。记为 $F \models X \rightarrow Y$
- **定义10.6** 设函数依赖集F，所有被F逻辑蕴涵的函数依赖称为F的**闭包**，记为 F^+
 - F^+ 可表示为： $F^+ = \{ X \rightarrow Y \mid \text{所有F蕴涵的FD } X \rightarrow Y \}$
- **定义10.8** 设关系模式R(U, F)， $U = \{A_1, A_2, \dots, A_n\}$ ， $X \subseteq U$ 。所有用公理推出的函数依赖 $X \rightarrow A_i$ 中 A_i 的属性集合称为**属性集X关于函数依赖集F的闭包**，记为 X_F^+

2. 函数依赖公理

- Armstrong公理系统就是一个有效而完备的公理系统，是模式分解算法的理论基础
 - 从一组函数依赖求得蕴含的函数依赖
 - 求给定关系模式的关键字

2. 函数依赖公理

□ Armstrong公理

- 设关系模式 $R(U, F)$, 并且 X 、 Y 、 Z 和 W 是 U 的子集
- A1 自反律 (Reflexivity)
若 $Y \subseteq X \subseteq U$, 则 $F \models X \rightarrow Y$
- A2 增广律 (Augmentation)
若 $X \rightarrow Y$ 且 $Z \subseteq U$, 则 $F \models XZ \rightarrow YZ$
- A3 传递律 (Transitivity)
若 $X \rightarrow Y$, $Y \rightarrow Z$, 则 $F \models X \rightarrow Z$

□ 三条很有用的推论

- 合成规则: 由 $X \rightarrow Y$, $X \rightarrow Z$, 则 $X \rightarrow YZ$
- 分解规则: 由 $X \rightarrow Y$ 及 $Z \subseteq Y$, 则 $X \rightarrow Z$
- 伪传递规则: 由 $X \rightarrow Y$, $YZ \rightarrow W$, 则 $XZ \rightarrow W$

求闭包的算法

□ 算法10.1 计算属性集 X 关于 F 的闭包 X_F^+

- 输入：模式 R 的属性全集 U ， U 上的函数依赖集 F ，属性集 X
- 输出：属性集 X 的闭包 X_F^+

□ 方法：计算 $X^{(i)}$ ($i=0, 1, \dots$)

- (1) 初值 $X^{(0)} = X$, $i=0$
- (2) $X^{(i+1)} = X^{(i)} \cup Z$; 其中, 属性集 $Z = \{ A \mid \text{存在 } V \rightarrow W \in F, V \subseteq X^{(i)} \text{ 且 } A \in W \text{ 而 } A \notin X^{(i)} \}$
- (3) 判断 $X^{(i+1)} = X^{(i)}$ 或 $X^{(i+1)} = U$ 是否成立, 若成立转(5)
- (4) $i=i+1$, 转(2)
- (5) 输出 X_F^+ 的结果 $X^{(i+1)}$

只要 F 中的函数依赖的左部属性包含在中间结果 $X^{(i)}$ 中, 就可以将没有出现在 $X^{(i)}$ 中的右部属性 A 并入 $X^{(i)}$ 中。 $X \rightarrow A$ 显然成立

求闭包的算法

□[例10-1] 设关系模式 $R(U, F)$, $U = \{A, B, C, D, E, G\}$, $F = \{AB \rightarrow C, BC \rightarrow D, ACD \rightarrow B, D \rightarrow EG, BE \rightarrow C, CE \rightarrow AG\}$, 求 $(BD)^+$

解: 令 $X = \{BD\}$, 初值 $X^{(0)} = \{BD\}$

- 在 F 中寻找左部是 BD 子集的函数依赖, $D \rightarrow EG$ 满足条件。结果为: $X^{(1)} = \{BDEG\}$ 。 $X^{(0)} \neq X^{(1)}$
- 在 F 中继续寻找左部是 $BDEG$ 子集的函数依赖, 得 $BE \rightarrow C$, C 不包含在 $BDEG$ 中, 结果为 $X^{(2)} = \{BCDEG\}$
- 在 F 中继续寻找左部是 $BCDEG$ 子集的函数依赖, 得 $BC \rightarrow D$, $CE \rightarrow AG$ 。这里仅有右部属性 A 是未出现在 $X^{(2)}$ 中的属性, 结果为 $X^{(3)} = \{ABCDEFG\}$
- $X^{(3)} \neq X^{(2)}$, 虽然 F 中还有未考察过的函数依赖, 但 $X^{(3)}$ 已包含了 R 中的所有属性, 结束
- 输出结果: $(BD)^+ = \{ABCDEFG\}$

3. 关系模式的规范化

- 定义10.15 如果关系模式R的每一个属性对应的域值都是不可再分的，则 $R \in 1NF$
- 定义10.17 如果一个关系模式 $R \in 1NF$ ，且所有非主属性都完全依赖于R的每个候选键，则 $R \in 2NF$
- 定义10.18 设 $R \in 1NF$ ，若在R中没有非主属性传递依赖于R的候选键，则关系模式 $R \in 3NF$

3. 关系模式的规范化

- 定义10.19 若 $R \in 1NF$ ，而且 R 中没有任何属性传递依赖于 R 中的任一关键字，则关系模式 R 属于Boyce-Codd范式 (BCNF)
- 定义10.20 设关系模式 $R \in 1NF$ ， F 是 R 上的函数依赖集，对于 F 中的每一个函数依赖 $X \rightarrow Y$ ，必有 X 包含 R 的一个候选键，则 $R \in BCNF$
 - 如果 $R \in BCNF$ ，则 R 上的每一个函数依赖中的每个决定因素都包含候选键
- 多值依赖定义、定律规则

3. 关系模式的规范化

□ 关系模式规范化的基本步骤

消除决定属性
集非码的非平
凡函数依赖

1NF

↓ 消除非主属性对候选键的部分函数依赖

2NF

↓ 消除非主属性对候选键的传递函数依赖

3NF

↓ 消除主属性对候选键的部分和传递函数依赖

BCNF

↓ 消除非平凡且非函数依赖的多值依赖

4NF

关系数据库设计理论的作用

- 数据库模式的设计是数据库系统开发的核心问题
 - 规范化理论为数据库设计提供了理论的指南和工具
 - 是设计关系逻辑数据模型的主要理论依据和方法
- 不能说规范化程度越高的关系模式就越好
 - 必须结合应用环境和现实世界的具体情况合理地选择数据库模式。规范化是范式的升高，而连接的本质是范式的降低
 - 大量的连接运算会使数据库的查询效率比较低，一般，数据库模式规范化到3NF或BCNF就可以了
- 模式分解
 - 无损连接分解
 - 分解保持依赖性

□数据库设计的过程

- (1) 需求分析
 - ◆ 准确了解与分析用户需求（包括数据与处理）
- (2) 概念结构设计
 - ◆ 通过对用户需求进行综合、归纳与抽象，形成一个独立于具体DBMS的概念模型
- (3) 逻辑结构设计
 - ◆ 将概念结构转换为某个DBMS所支持的数据模型，并对其进行优化

□数据库设计的过程

- (4) 数据库物理设计
 - ◆ 为逻辑数据模型选取一个最适合应用环境的物理结构（包括存储结构和存取方法）
- (5) 数据库实施
 - ◆ 用DDL定义数据库结构、组织数据入库
 - ◆ 编制与调试应用程序、数据库试运行
- (6) 数据库运行和维护



- 概念设计经常采用的策略是自底向上
- (1) 设计局部E-R模型
- (2) 设计全局E-R模型
 - 合并：
 - ◆ 解决各分E-R图之间的冲突，将所有局部E-R图集成为一个全局E-R图
 - ◆ 冲突主要有三类：属性冲突、命名冲突、结构冲突
 - 修改与重构：
 - ◆ 消除不必要的冗余，生成基本E-R图
- (3) 验证整体概念结构



- 将概念结构转化为关系（网状、层次）模型
 - 将E-R图转换为关系模型：
 - ◆ 将实体、实体的属性和实体之间的联系转化为关系模式
- 对数据模型进行优化

E-R模型向关系模型的转换

- 一个实体转换为一个关系模式。实体的属性就是关系的属性，实体的标识符就是关系的码
- 对于实体间的联系有以下不同的情况
 - 一个1:1联系可以转换为一个独立的关系模式，也可以与任意一端所对应的关系模式合并
 - 一个1:n联系可以转换为一个独立的关系模式，也可以与n端所对应的关系模式合并
 - 一个m:n联系转换为一个独立的关系模式
 - 三个或三个以上实体间的一个多元联系可以转换为一个关系模式
 - 具有相同码的关系模式可以合并

第12章 数据库编程



- 1. 存储过程
- 2. 用户定义函数
- 3. 触发器

1. 存储过程

□ 存储过程的作用

- ① 接受输入参数并以输出参数的格式向调用过程或批处理返回多个值
- ② 包含在数据库中执行操作的编程语句
- ③ 向调用过程或批处理返回状态值，以指明成功或失败（以及失败的原因）
- ④ 可以使用EXECUTE语句执行存储过程

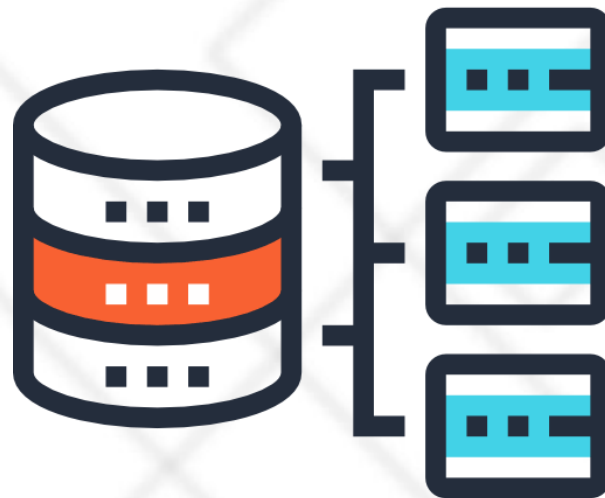
□ 存储过程与函数不同，因为存储过程不返回取代其名称的值，也不能直接在表达式中使用

2. 用户定义函数

- 函数也是一种存储过程，只不过它能返回值，返回值可以是单个标量值或结果集
 - 返回单个数据值的函数称为**标量函数**
 - 返回结果集的函数称为**表值函数**
 - ◆ 内嵌表值函数
 - 内嵌表值函数返回一个单条SELECT语句产生的结果的表
 - 内嵌表值函数可以看成带参数的视图
 - ◆ 多语句表值函数
 - 多语句表值函数返回一个由一条或多条Transact-SQL语句建立的表



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



That's all!

Q & A

北京理工大学计算机学院 段春晖

duanch@bit.edu.cn